Scaling notes: imperfect reward (Sections 4 and 6)

Bo Xu

These are my notes on scaling the imperfect-reward side, where one Observer has a fixed attention budget and only reports a reward back to an Actor with probability $\theta(\mu_j) = 1 - e^{-c\mu_j}$ (it spends the budget μ on the Actors it rates highest). I followed the ladder we agreed on: scale states first, then actors, then both. Every comparison keeps the step count fixed, so it's at equal compute. Short version: states alone and actors alone are fine; doing both at once works up to a point and then falls off a cliff, and I think I know why.

1 States alone (1 Actor, 1 Observer)

The state is a cell on a 2-D grid and the Actor sees its coordinates, so nearby cells have similar best actions (I avoided random per-state actions on purpose, as you suggested). One thing tripped me up first: with all-positive rewards, plain REINFORCE just reinforces whatever it samples first and collapses. Subtracting a running-mean baseline fixed it, and I left it as an opt-in flag so the older cases don't change.

After that it just works. Going from 4 up to 1024 states, the clear cells are always right; the only misses are the thin band of cells sitting on a region boundary, where the best action is genuinely ambiguous. Accuracy doesn't keep dropping with size (0.90, 0.90, 0.93 at 64, 256, 1024 states) because that boundary band is a smaller share of a finer grid.

states	greedy acc	mean P(correct)	representative P(correct)
4	1.000	1.000	1.000
16	1.000	0.999	1.000
64	0.901	0.899	1.000
256	0.897	0.893	1.000
1024	0.928	0.921	1.000

Table 1: States alone: stays high all the way to 1024; the well-inside cells stay perfect. So state count by itself isn't the bottleneck.

2 Actors alone (1 Observer, many Actors)

Now the opposite: one trivial state, but a growing crowd of Actors, two of which are good and the rest are noisy distractors. The question is whether the fixed budget can still land on the two good ones as the crowd grows.

It keeps *finding* them the whole way: the two good Actors are almost always the top-ranked ones (top-k precision ≈ 1.0, even at 256). What it loses is the ability to put the *whole* budget on them. Each bad Actor only skims a tiny slice, but with hundreds of them the leak adds up, so the share of budget on the good Actors drops from ≈ 0.99 at 4 Actors to ≈ 0.59 at 256. Ranking survives, allocation dilutes – which is just what the equations say, not a bug.

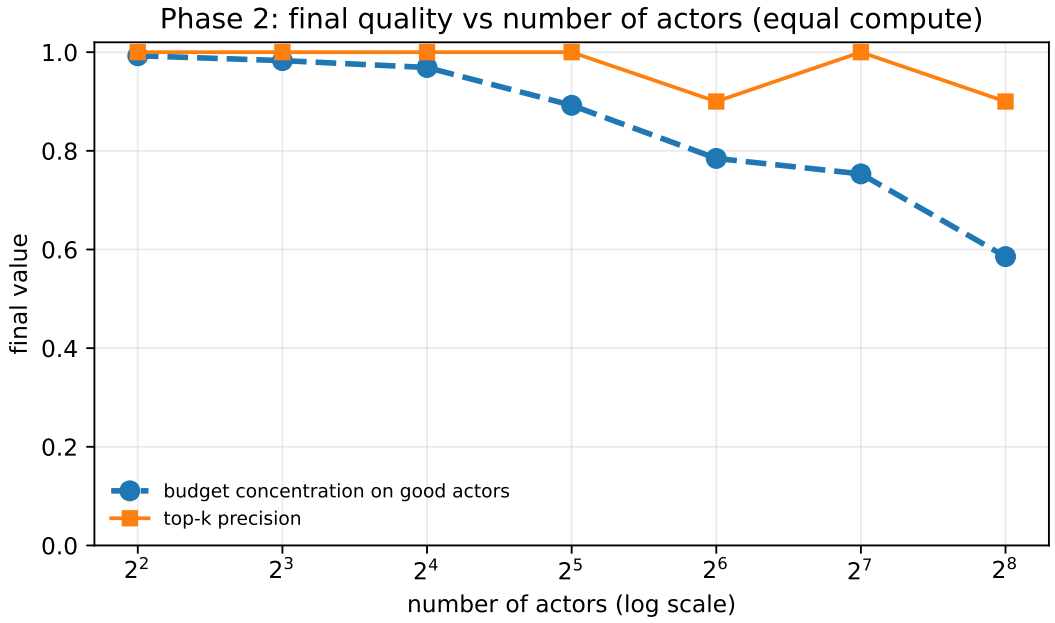


Figure 1: Actors alone: the Observer keeps ranking the good Actors at the top (top-k precision flat near 1.0), but a fixed budget gets spread thinner as the crowd grows.

3 What happens when you increase the budget B?

After sharing the actors-alone results, Peter asked whether keeping B fixed was the right choice, or whether scaling B with the crowd would help. I ran two follow-up experiments.

Case A – proportional scaling ($B = N/2$, 2 good Actors out of N).

N	B	conc	top-2	$B \times$ conc
4	2	1.000	1.00	2.0
16	8	0.583	1.00	4.7
64	32	0.166	0.90	5.3
256	128	0.046	1.00	5.9

Table 2: Scale N with $B = N/2$: concentration ratio collapses, but the absolute reports to good Actors ($B \times$ conc) actually grows.

Case B – fixed $N = 32$, increasing B.

B	conc	top-2	$B \times$ conc
2	0.948	1.00	1.9
4	0.719	1.00	2.9
8	0.507	1.00	4.1
16	0.344	1.00	5.5
32	0.242	1.00	7.8

Table 3: Fix $N = 32$, vary B: top-2 precision stays at 1.0 throughout, but concentration falls monotonically with B.

The same picture emerges in both cases: the Observer always *identifies* the good Actors correctly (top-k stays near 1.0), but a larger B makes it harder to concentrate the whole budget on just two of them. The softmax has a fixed utility gap to work with; raising B does not widen that gap, so the relative concentration shrinks. At the same time, $B \times$ conc grows, meaning good Actors do receive more absolute reward feedback – so the effect on learning is ambiguous. My guess is that the sharpness parameter would need to scale with B to keep concentration stable, but I have not tested that.

4 Both at once

Here I combine the two: the grid task (many states) *and* a crowd of Actors. Two of them can see the state and learn a real policy; the rest are "blind" (their input is zeroed) and can only learn one fixed action. I grow both axes together at a fixed budget. The "system score" below is just accuracy weighted by how much budget each Actor got, so it's only high when the good Actors learn *and* the Observer follows them.

states	actors	good-actor acc	budget on good	system score
16	8	1.000	0.942	0.956
36	16	0.907	0.832	0.803
64	32	0.638	0.531	0.478
100	64	0.302	0.052	0.258
144	128	0.262	0.000	0.250

Table 4: Both at once: fine up to about 64 states / 32 actors, then it falls off a cliff to chance (0.25 with 4 actions).

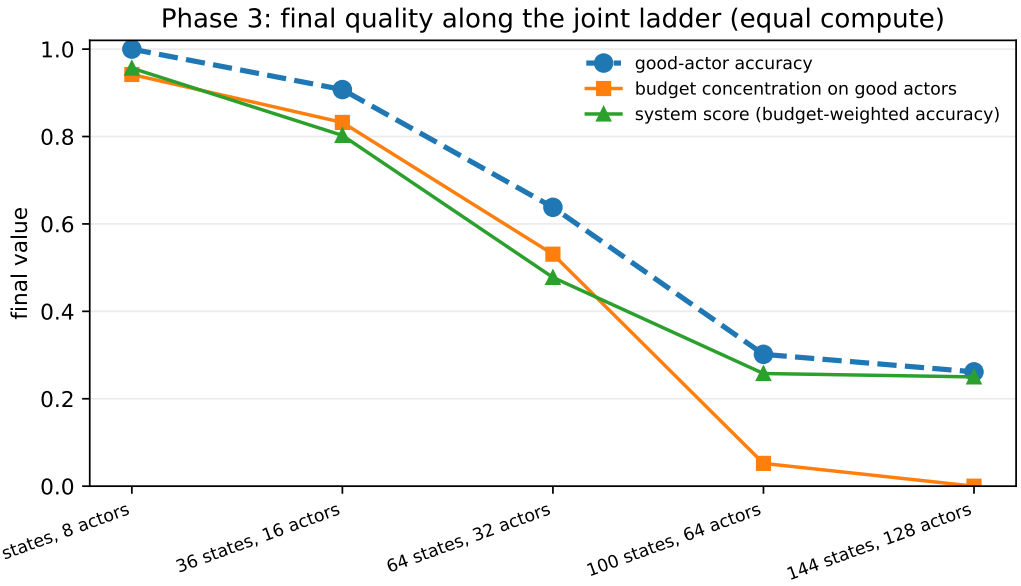


Figure 2: Both at once: graceful for the first few rungs, then everything (accuracy, budget concentration, system score) drops together past 32 actors.

5 Why it falls off the cliff (a question for you)

This is the one place I'm stuck and would like your read. The thing I can measure is that the budget starts spread evenly, $\mu_j = B/N$, so the chance an Actor's reward gets reported back is $1 - e^{-cB/N}$ – about 0.39 at 8 Actors but only 0.03 at 64. So at large N a good Actor gets very few reports early on, and it looks like its utility never gets a chance to separate from the blind crowd.

What I don't understand is why it collapses so sharply – to chance – rather than just degrading more gently like the actors-alone case did. Is it the effect I suspect (too few early reports, so nobody learns and nobody gets followed), or is something else going on? And how would you suggest getting around it? I didn't want to start trying fixes before checking with you what the right way to think about it is.

6 Where this leaves us

States alone: fine to 1024. Actors alone: ranking stays correct to 256, budget just dilutes. Both together: fine to about 64 states / 32 actors, then the cliff.

Everything's committed with tests, and there are longer per-phase notes in the repo if you want the details.